



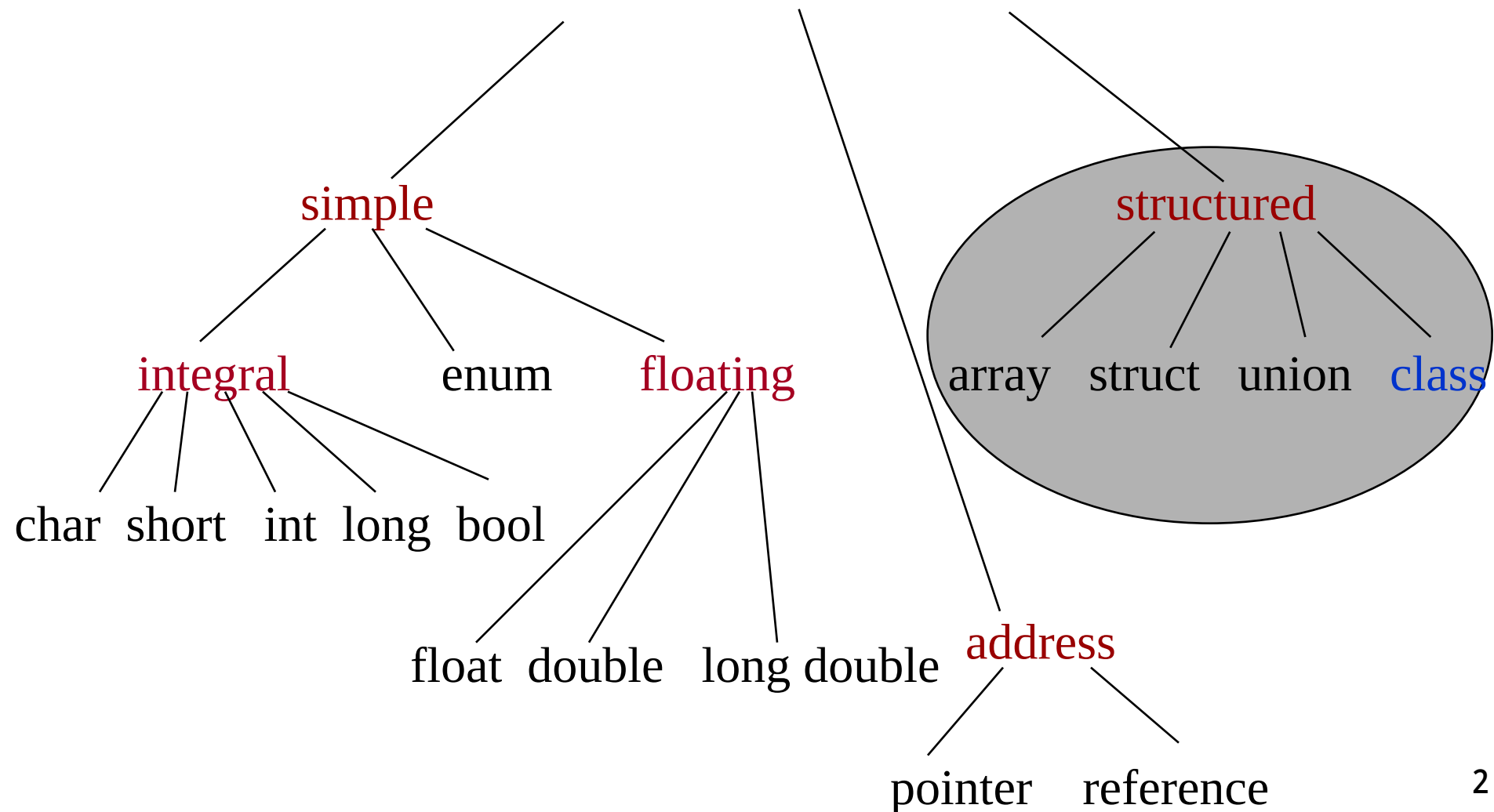
程序设计原理

class类

任课教师：喻 梅
于瑞国
王建荣
李雪威
赵满坤



C++ Data Types





C++ classType

- Facilitates **re-use** of C++ code for an ADT
- Software that uses the class is called a **client**
- Variables of the class type are called **class objects** or **class instances**
- Client code uses class's public member functions to manipulate class objects



Client Code Using Time

```
#include "time.h" // Includes specification of the class
using namespace std;

int main ()
{
    Time currentTime; // Declares two objects of Time
    Time endTime;
    bool done = false;

    currentTime.Set (5, 30, 0);
    endTime.Set (18, 30, 0);
    while (! done)
    {
        . . .

        currentTime.Increment ();
        if (currentTime.Equal (endTime))
            done = true;
    };
}
```



class type Declaration

The class declaration creates a data type and names the members of the class

It **does not allocate memory for any variables of that type!**

Client code still needs to declare class variables



Remember ...

- Two kinds of class members:
data members and **function members**
- Class members are private by default
- Data members are generally private
- Function members are generally declared public
- Private class members can be accessed only by the class member functions (and friend functions), not by client code



Class Scope

- **Every function member can access any other members (function or data) no matter in what order the members are placed in the class body block**
- **Declaration-before-use rule does not apply to class members**



Aggregate class Operations

- **Built-in operations valid on class objects are**
Member selection using dot (.) operator ,
Assignment to another class variable using (=),
Pass to a function as argument
(by value or by reference),
Return as value of a function
- Other operations can be defined as class member functions



Separate Specification and Implementation

```
// Specification file "time.h"  
// Specifies the data and function members  
class Time  
{  
public:  
    . . .  
  
private:  
    . . .  
};
```

```
// Implementation file "time.cpp"  
// Implements the Time member functions  
{  
    . . .  
}
```



Implementation File for Time

```
// Implementation file "time.cpp"
// Implements the Time member functions.
#include "time.h" // Also must appear in client code
#include <iostream>

. . .

bool Time::Equal(/* in */ Time otherTime) const
// Postcondition: Return value == true,
// if this time equals otherTime,
// otherwise == false
{
    return ((hrs == otherTime.hrs)
        && (mins == otherTime.mins)
        && (secs == otherTime.secs));
}

. . .
```



Should be familiar ...

- The member selection operator (.) selects either data members or function members
- Header files **iostream** and **fstream** declare the **istream**, **ostream**, and **ifstream**, **ofstream** I/O classes
- Both **cin** and **cout** are class objects and **get** and **ignore** are function members

```
cin.get (someChar);  
cin.ignore (100, '\n');
```

- These statements declare **myInfile** as an instance of class **ifstream** and invoke function member **open**

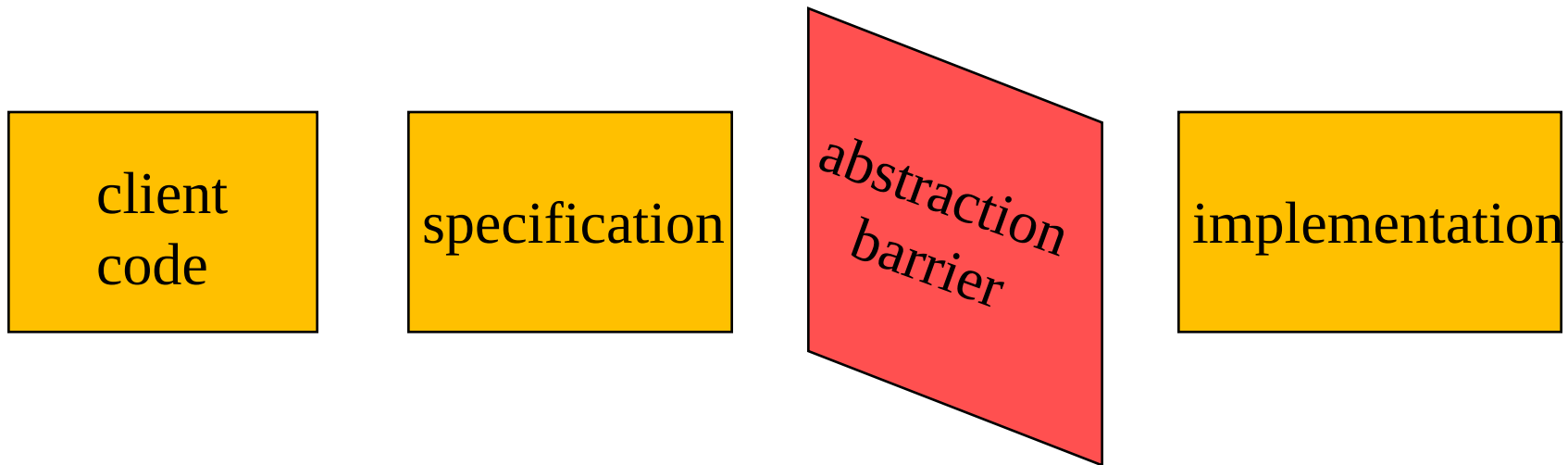
```
ifstream myInfile;  
myInfile.open ("mydata.dat");
```



Information Hiding

Information hiding - Class implementation details are hidden from the client's view

Public functions of a class provide the **interface** between the client code and the class objects





Selection and Resolution

- C++ programs typically use several class types
- Different classes can have member functions with the same identifier, like Write()
- Member **selection operator** is used to determine the object to whom member function Write() is applied

currentTime.Write(); // **Class Time**

numberZ.Write(); // **Class ComplexNumber**

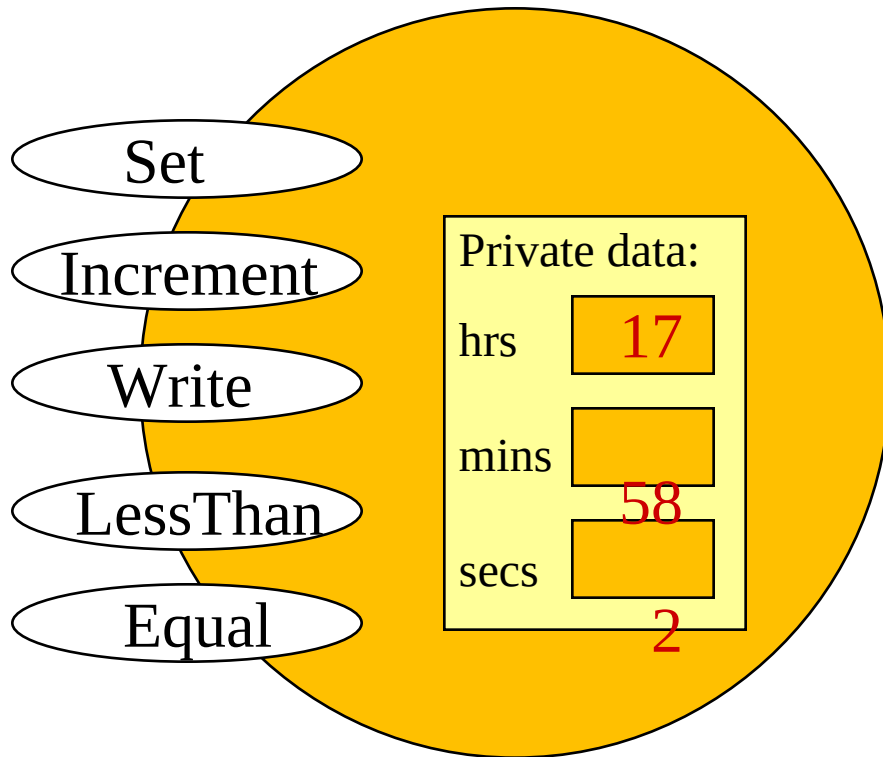
- In the implementation file, the **scope resolution operator** is used in the heading before the function member's name to specify its class

```
void Time::Write ()  const
{
    . . .
}
```

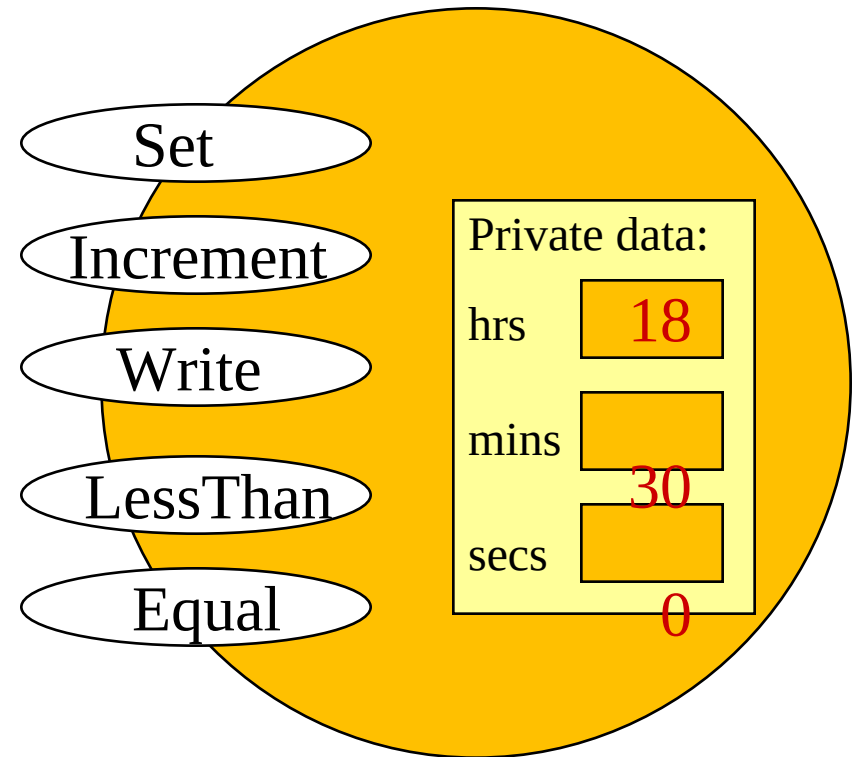


Time Class Instance Diagrams

currentTime



endTime





Use of const with Member Functions

- When a member function does not modify the private data members, use **const** in both the function prototype (in specification file) and the heading of the function definition (in implementation file)

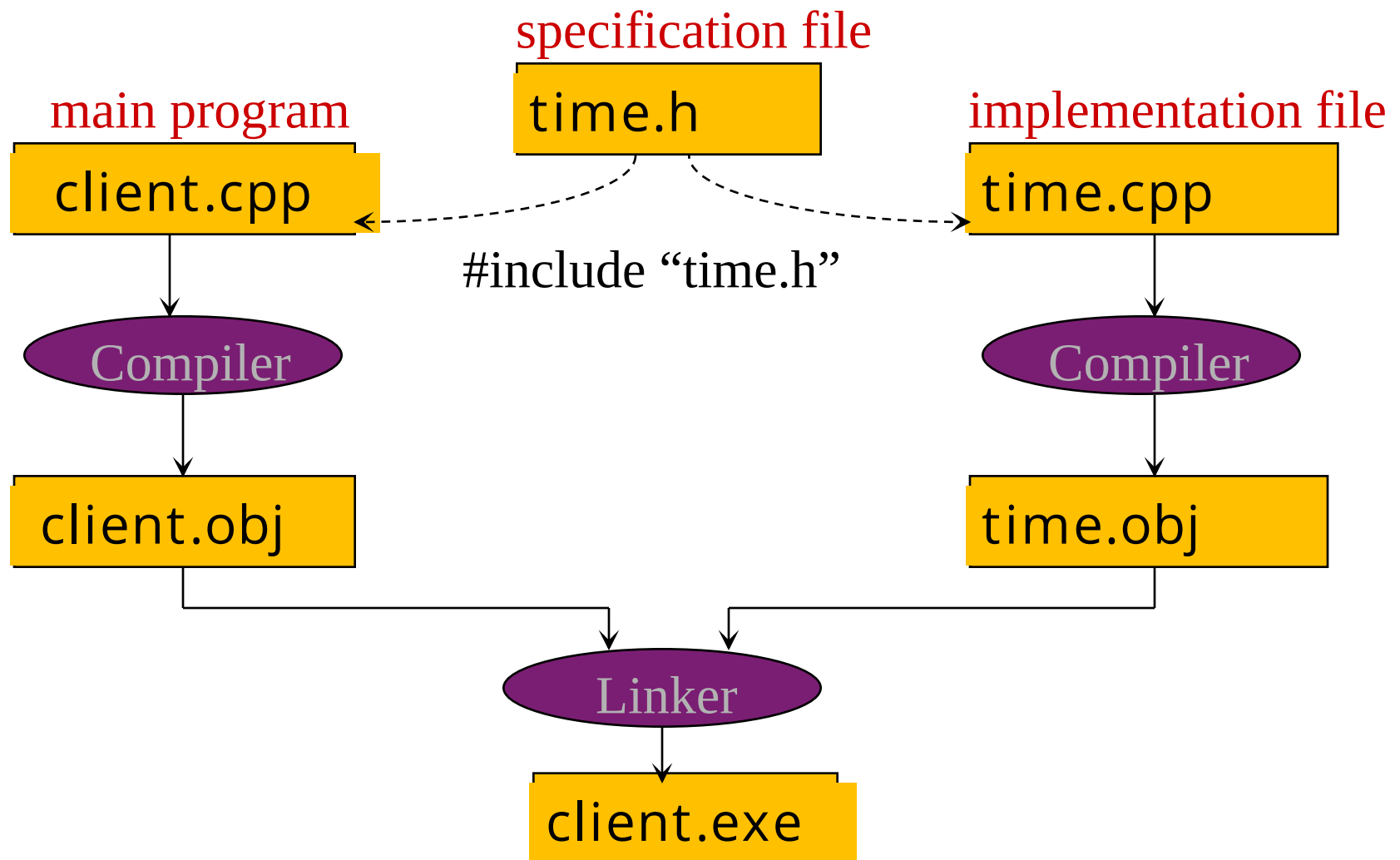


Example: Using const with a Member Function

```
void Time::Write () const
// Postcondition: Time has been output in form
// HH:MM:SS
{
    if (hrs < 10)
        cout << '0';
    cout << hrs << ':';
    if (mins < 10)
        cout << '0';
    cout << mins << ':';
    if (secs < 10)
        cout << '0';
    cout << secs;
}
```



Separate Compilation and Linking of Files





Avoiding Multiple Inclusion of Header Files

- Often several program files use the same header file containing typedef statements, constants, or class type declarations--but, it is a **compile-time error to define the same identifier twice within the same namespace**
- This preprocessor directive syntax is used to avoid the compilation error that would otherwise occur from multiple uses of #include for the same header file

```
#ifndef Preprocessor_Identifier  
#define Preprocessor_Identifier  
:  
:  
#endif
```



Example: Using Preprocessor Directive `#ifndef`

```
// time .h  
// Specification file
```

```
#ifndef TIME_H  
#define TIME_H
```

```
class Time  
{  
    public:  
    ...  
  
    private:  
    ...  
};  
#endif
```

For compilation the class declaration in
File time.h will be included only once

```
// time .cpp  
// IMPLEMENTATION FILE
```

```
#include "time.h"
```

```
...
```

```
// client.cpp  
// Appointment program
```

```
#include "time.h"
```

```
int main (void)  
{  
    ...  
}
```



Class Constructors

- A **class constructor** is a member function whose purpose is to initialize the private data members of a class object
- The name of a constructor is always the name of the class, and there is no return type for the constructor
- A class may have several constructors with different parameter lists
- A constructor with no parameters is the **default** constructor
- A constructor is **implicitly invoked** when a class object is declared--if there are parameters, their values are listed in parentheses in the declaration



Class Constructors

```
class Time    // Time.h
{
public :      // 7 function members
    void Set(int hours, int minutes, int seconds);
    void Increment();
    void Write() const;
    bool Equal(Time otherTime) const;
    bool LessThan(Time otherTime) const;

    // Parameterized constructor
    Time (int initHrs, int initMins, int initSecs);
    // Default constructor
    Time();
    //~Time();
private :    // 3 data members
    int hrs;
    int mins;
    int secs;
};
```



Implementation of Time Default

```
Time::Time ()  
// Default Constructor  
// Postcondition:  
// hrs == 0 && mins == 0 && secs == 0  
{  
    hrs = 0;  
    mins = 0;  
    secs = 0;  
}
```



Parameterized Constructor

```
Time::Time( /* in */ int  initHrs,  
            /* in */ int  initMins,  
            /* in */ int  initSecs)  
  
// Constructor  
// Precondition:  
//    0 <= initHrs <= 23 && 0 <= initMins <= 59  
//    0 <= initSecs <= 59  
// Postcondition:  
//    hrs == initHrs && mins == initMins  
//    && secs == initSecs  
{  
    hrs = initHrs;  
    mins = initMins;  
    secs = initSecs;  
}
```

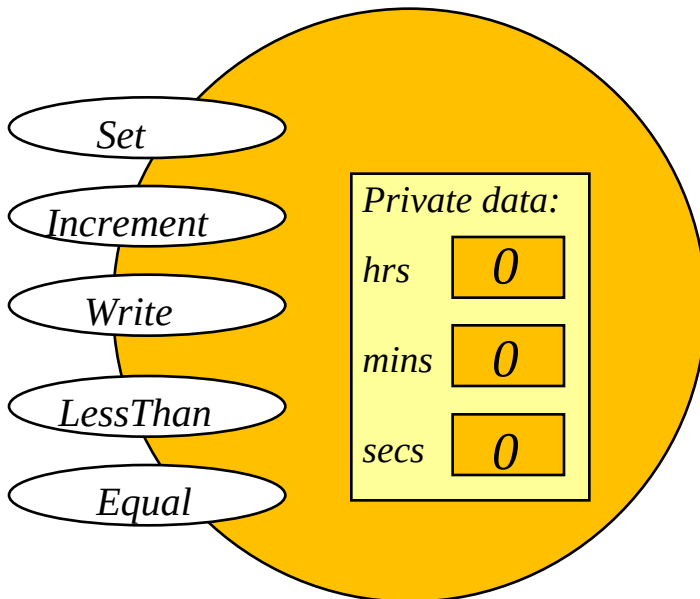


Automatic invocation of constructors occurs

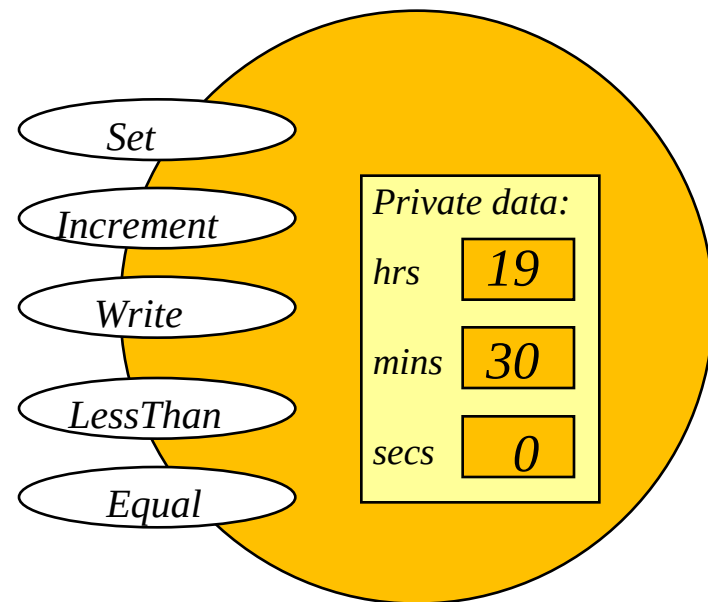
Time departureTime; // Default constructor invoked

Time movieTime (19, 30, 0); // Parameterized constructor

departureTime



movieTime





Exercise

Print a Point Class

要求编写一个类，来代表二维坐标系上的点。

要求实现函数：1. 打印该点坐标 2. 求两点之间距离 3. 返回一个点对象，为该点相对于原点的(0,0)的对称点 4. 判断点是否在给定圆内。

Input

第一行输入两个整数 x, y ，代表点的坐标 (x, y) 。

第二行输入一个正整数 $case$ ，代表测试数据的组数。

对于每一组 $case$ ，第一行输入一个数，代表要调用哪个函数，1 2 3 4对应题干函数。

对于函数1，无后续输入

对于函数2，输入两个整数 p, q ，代表另一个点的坐标 (p, q)

对于函数3，无后续输入

对于函数4，输入两个整数 p, q, r ， (p, q) 代表圆心坐标， r 代表圆的半径(d 点在圆上不算做在圆内)

Output

对于函数1，打印该点坐标，用空格隔开

对于函数2，打印两点间距离（误差不超过 10^{-5} ）

对于函数3，打印结果点坐标，用空格隔开

对于函数4，，若点在圆内输出“Yes”，否则输出“No”



Exercise

Constraints

输入的数均不超过1000

Sample Input

1 1

4

1

2

0 1

3

4

0 0 3

Sample Output

1 1

1

-1 -1

Yes



Exercise

Constraints

输入的数均不超过1000

Sample Input

1 1

4

1

2

0 1

3

4

0 0 3

Sample Output

1 1

1

-1 -1

Yes

```
class Point{
private:
    double x, y;

public:
    Point(double xx=0, double yy=0){
        x=xx;
        y=yy;
    }
    void print(){
        cout<<x<<" "<<y<<endl;
    }
    double distance(Point& b){
        return sqrt((x-b.x)*(x-b.x)+(y-b.y)*(y-b.y));
    }
    Point syn(){
        return Point(-x,-y);
    }
    bool circle(double p,double q,double r){
        Point pt1(x,y),pt2(p,q);
        if (pt1.distance(pt2)<r) return true;
        else return false;
    }
}pt;
```



Exercise

Print a Circle Class

要求编写一个类，来代表二维坐标系上的圆。

要求实现函数：1. 打印圆心坐标 2. 求圆的面积 3. 求圆的周长 4. 判断两圆是否相切 (包括内切和外切)。

Input

第一行输入三个整数 x, y, r ，代表圆心的坐标 (x, y) ，半径 r 。

第二行输入一个正整数 $case$ ，代表测试数据的组数。

对于每一组 $case$ ，第一行输入一个数，代表要调用哪个函数，1 2 3 4对应题干函数。

对于函数1，无后续输入

对于函数2，无后续输入

对于函数3，无后续输入

对于函数4，输入两个整数 $p, q, r1$ ， (p, q) 代表第二个圆圆心坐标， $r1$ 代表第二个圆半径

Output

对于函数1，打印圆心坐标，用空格隔开

对于函数2，打印圆的面积，误差不超过 10^{-5}

对于函数3，打印圆的周长，误差不超过 10^{-5}

对于函数4，，若点在两圆相切输出“Yes”，否则输出“No”



Exercise

Constraints

输入的数均不超过1000

Sample Input

0 0 1

4

1

2

3

4

2 0 1

Sample Output

0 0

3.14159265

6.2831853

Yes



Exercise

Print a MyString Class

该问题需要实现一个MyString类。需要包含以下成员函数。

MyString(const char*) 将字符串构建为一个MyString实例

MyString.length() 返回字符串的长度

MyString.find(const char* find_str) 查询实例中是否含有find_str，如果含有，则返回首位置长度，不含则返回-1。

Input

第一行包含一个只有小写字母的字符串，表示需要将这个字符串构建为MyString实例

第二行包含一个整数N，表示询问次数

之后N行，如果输入1，则返回字符串的长度。

如果输入2 str，则返回在实例中查询str的结果。其中str为只包含小写字母的字符串。

Output

对于每个询问，输出询问结果，结果占一行。



Exercise

Constraints

输入的字符串长度均不超过10000

Sample Input

hellomysting

4

1

2 hello

2 hollw

2 my

Sample Output

13

0

-1

5